

第 20 章 SSH 客户端、服务端与密钥认证

从“连得上”走到“身份可信、认证可证、配置可回退”：把客户端、服务端、密钥和证据放进同一条链。

对象模型 · 操作语义 · 验证 · 诊断 · 经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：客户端先确认服务器是谁，服务器再确认用户是谁；配置、权限和进程状态只是这条双向信任链的输入，独立新连接才是功能终态。

01解析目标确认别名、真实主机、用户与端口

02建立传输区分名称、路由、过滤与监听

03验证主机用可信指纹约束 known_hosts

04选择身份确认私钥、agent 与客户端匹配

05服务端授权检查有效配置、authorized_keys 与路径可信性

06分层验收证明方法、远端用户、主机和策略终态

专题地图

- 知识专题 SSH 连接阶段与双向身份
- 知识专题 主机密钥与 known_hosts
- 操作专题 用户密钥、authorized_keys 与 ssh-copy-id
- 操作专题 客户端匹配与 ssh-agent
- 知识专题 sshd 配置层与有效值
- 操作专题 安全修改、reload 与第二连接
- 诊断专题 从 ssh -vvv 到服务端日志
- 经典任务 建立密钥认证并安全收紧 sshd

阅读时持续回答

1. 当前失败停在传输、主机身份还是用户认证？
2. 客户端最终连接到谁、使用哪个账号和端口？
3. known_hosts 记录的是哪类身份？
4. 私钥、公钥和 authorized_keys 分别位于哪一端？
5. 客户端是否真的发送了预期密钥？
6. sshd 对目标连接实际采用了哪些配置？
7. 权限、所有权和 SELinux 标签分别证明什么？
8. 当前证据能否证明独立新连接的最终功能？

章节边界：网络与路由留给第 18 章，名称解析留给第 19 章，firewalld 留给第 21 章，持久 SELinux 规则留给第 29 章。本章只建立 SSH 所需的接口和验收位置。

第 20 章 SSH 客户端、服务端与密钥认证

远程登录看起来只是一条 `ssh user@host`，但真正完成的系统工作远不止“连接一个端口”。客户端必须先把别名还原为真实目标并建立传输，再验证服务器主机身份；服务器随后才判断来访者是否可以作为目标用户登录；认证通过后，双方才创建交互 Shell 或远程命令通道。只要把这些阶段混在一起，就会出现高频误判：端口可达便开始重建用户密钥，看到密码提示便忽略主机身份，登录成功却没有发现公钥失败后回退到了密码，或修改 `sshd` 后只看服务仍为 `active` 就宣布完成。

本章以“双向身份链”为中心组织对象。客户端侧保存目标、主机信任、用户私钥和 agent 状态；服务端侧保存主机私钥、用户授权、配置层、路径权限和运行状态。讲义将持续区分三种证据：配置文本说明管理员写了什么，有效配置与进程状态说明软件当前采用什么，独立功能测试才说明指定用户能否按指定方法连接到指定主机。

本章不完整讲解地址、路由、DNS、防火墙或 SELinux 策略。遇到这些对象时，只指出 SSH 链路需要读取什么证据，并把完整修改方法交还给相邻章节。这样可以避免用“关闭防火墙、关闭 SELinux、删除整个 `known_hosts`”之类捷径掩盖真正的失败层。

概念

主机身份回答“正在连接的服务器究竟是谁”。服务端使用 `/etc/ssh/ssh_host_*_key` 中的主机私钥证明身份，客户端将核验后的主机公钥记录在 `known_hosts`。主机身份验证发生在用户认证之前；即使账号密码或用户私钥正确，也不应把凭据交给一个尚未确认身份的对端。可观察证据是首次连接指纹、既有 `known_hosts` 记录和主机密钥变化警告。

概念

用户身份回答“来访者是否有权作为目标账号登录”。公钥认证中，客户端持有私钥并完成签名，服务端目标账号只保存相应公钥；密码认证则由服务端验证口令。用户身份与主机身份是两条独立信任链：`authorized_keys` 不能证明服务器是谁，`known_hosts` 也不能授权某个用户登录。

概念

`known_hosts` 是客户端保存的主机信任记录，而不是“连接缓存”。首次连接时，记录必须来自可信指纹核验；后续密钥变化可能来自重装、轮换、目标错误或中间人攻击。删除条目只能清除旧记录，不能证明新密钥可信，因此安全顺序始终是先调查变化，再核对新指纹，最后只更新相关目标。

概念

`authorized_keys` 是服务端账号的授权入口。一行公钥表示该账号允许对应私钥持有者尝试认证，但文件存在并不等于认证一定成功：`sshd` 还要按有效配置找到该文件，并确认 `home`、`ssh`、授权文件的所有权和可写权限可信，RHEL 上还可能受到 SELinux 标签影响。

概念

客户端配置匹配把命令行目标映射为真实主机、用户、端口和候选身份。`Host` 可以是别名或模式，`HostName` 才是实际目标；命令行、用户配置和系统配置共同参与取值。不能只靠肉眼阅读某一段配置，应该用 `ssh -G` 查看完成 `Host` 与 `Match` 计算后的最终选择。

概念

`sshd` 配置层包含主配置、`Include` 引入的 drop-in 和可能改变特定连接的 `Match` 块。管理员看到的某一行不一定是守护进程采用的值；`sshd -t` 只能证明配置可解析，`sshd -T` 和 `sshd -T -C ...` 才分别展示全局及特定连接上下文的有效配置。

概念

认证阶段位于传输建立和主机验证之后。客户端可能依次尝试多种方法，公钥失败后仍可能回退到 `password` 或 `keyboard-interactive`。因而“进入了 Shell”不能单独证明公钥认证成功；需要限制回退、读取 `ssh -vvv` 的关键阶段，并用远端 `id`、`hostname` 和退出状态完成终态确认。

操作语义速查区

这里先建立接口地图。后续专题解释为什么这样用、操作改变哪个对象，以及每条证据能够证明到哪一层。

ssh

SYNOPSIS

```
ssh [-v] [-F configfile] [-i identity_file] [-J destination]
    [-p port] [-o option] [user@]hostname [command [argument ...]]
```

建立 SSH 客户端会话，完成目标选择、主机验证、用户认证，并请求交互 Shell 或远程命令。

`-i identity_file`

显式指定候选私钥；它不自动限制 agent 中其他身份。

`-p port`

连接非默认服务端端口；端口放行和 SELinux 端口类型分别交给后续章节。

`-F configfile`

使用指定客户端配置文件，适合隔离测试。

`-J destination`

通过跳转主机建立连接；本章只认识接口，不展开复杂跳板拓朴。

`-v / -vv / -vvv`

逐级增加调试信息。诊断时只抓连接目标、主机密钥、密钥 offer、服务端接受和最终方法等关键阶段。

ssh-keygen

SYNOPSIS

```
ssh-keygen -t type [-b bits] [-C comment] [-f output_keyfile]
ssh-keygen -lf public_key
ssh-keygen -F host | ssh-keygen -R host
```

生成和管理用户或主机密钥，并以指纹识别密钥材料；同一工具也可查询或删除 `known_hosts` 中的目标记录。

-t type

显式选择密钥算法。Ed25519 不适用于 FIPS 模式；不要把版本默认值当成稳定考试结论。

-b bits

为支持位数设置的算法指定长度，例如 RSA。

-f path

指定输出或输入文件，避免覆盖既有默认密钥。

-C comment

写入便于审计和轮换的注释，不参与加密验证。

-l

显示公钥或已知主机条目的指纹。

-F / -R

查找或移除指定主机记录；移除前必须完成可信核验。

ssh-copy-id

SYNOPSIS

```
ssh-copy-id [-i identity_file.pub] [user@]hostname
```

使用已有可用认证路径登录远端，并把指定公钥追加到目标账号的授权位置。它不会复制私钥，也不会自动修改 `sshd` 认证策略。

-i identity_file.pub

明确选择要安装的公钥文件。指向私钥是对象方向错误。

user@hostname

决定公钥授权给哪个服务端账号；连接成功后仍要核对远端身份和授权文件状态。

ssh-agent / ssh-add

SYNOPSIS

```
eval "$(ssh-agent -s)"
ssh-add [-t life] private_key
ssh-add -l | ssh-add -L | ssh-add -d private_key | ssh-add -D
```

agent 是客户端本地签名服务；`ssh-add` 管理当前 agent 中已经解锁的身份。agent 中存在身份不代表服务端账号已经授权。

SSH_AUTH_SOCKET

当前 Shell 访问 agent 的 Unix socket；变量为空或 socket 不可达时，客户端无法使用该 agent。

-l / -L

分别列出指纹或公钥内容。

-d / -D

移除一个或全部 agent 身份；不删除磁盘私钥，也不撤销服务端授权。

-t life

为缓存身份设置有限生命周期。

客户端匹配：ssh_config 与 ssh -G

SYNOPSIS

```
ssh -G [-F configfile] destination

Host alias
  HostName server.example.com
  User operator
  Port 22
  IdentityFile ~/.ssh/operator_key
  IdentitiesOnly yes
```

客户端配置把别名和匹配条件转换为实际连接参数；`ssh -G` 输出计算后的结果，适合证明客户端最终选择。

Host

定义别名或匹配模式；更具体的规则应优先于通用规则。

Match

按连接条件应用客户端配置；使用时必须通过有效配置而不是肉眼猜测。

HostName / User / Port

分别指定实际目标、远端账号和端口。

IdentityFile

增加候选身份文件。

IdentitiesOnly yes

限制客户端使用显式配置的身份，减少多密钥误选。

服务端配置：sshd_config / drop-in / Match

SYNOPSIS

```
Include /etc/ssh/sshd_config.d/*.conf
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
AuthorizedKeysFile .ssh/authorized_keys
Match User operator
...
```

服务端配置决定监听、认证方法、账号边界和授权文件位置。主配置、drop-in 顺序与 `Match` 上下文共同决定最终值。

Include

引入其他配置源；必须检查它在主文件中的实际位置。

Match

让一部分指令只作用于指定用户、组或地址。

PermitRootLogin

控制 root 远程登录边界。

PasswordAuthentication

控制 SSH password 方法；不能自动代表 keyboard-interactive 也已关闭。

PubkeyAuthentication

控制用户公钥认证。

AuthorizedKeysFile

指定服务端从何处读取用户授权公钥。

验证链：sshd -t/-T、systemctl、ss、journalctl

SYNOPSIS

```
sshd -t
sshd -T
sshd -T -C user=name,host=host,addr=remote,laddr=local,lport=22
systemctl reload sshd
ss -lntp
journalctl -u sshd --since '-5 min' --no-pager
```

这一组命令分别证明配置可解析、有效值、特定连接上下文、服务应用、监听状态和服务端认证证据；它们必须分层使用。

sshd -t

证明配置可解析及关键文件满足基本检查；不证明已 reload。

sshd -T

输出全局有效配置。

sshd -T -C ...

模拟连接条件并应用 `Match`。

systemctl reload sshd

让新连接采用新配置；执行前必须先通过静态检查并保留旧会话。

ss -lntp

证明本机监听；不证明外部防火墙已放行。

journalctl -u sshd

解释服务端接受或拒绝；应与客户端最小复现按时间关联。

[知识专题] SSH 连接不是一步：先连接、再认主机、再认用户

排错 SSH 时，最有效的切入不是先改配置，而是先判断失败停在哪个阶段。名称解析、TCP 连接、算法协商、主机身份、用户认证和会话创建使用不同对象，也产生不同证据。客户端 `ssh -vvv` 的价值正是在于把一条“连接失败”拆回这些阶段。

① [知识点] TCP 可达只证明传输入口存在

客户端首先把目标名称解析为地址，然后尝试连接指定端口。`Connection timed out` 通常表示请求没有获得及时响应，可能涉及路由、过滤、地址错误或中间网络；`Connection refused` 通常表示目标返回拒绝，常见于该地址端口没有监听。它们都发生在用户认证之前，不能通过重建用户密钥来修复。

网络地址和路由的完整调查属于“NetworkManager、IPv4 地址与路由”；名称解析属于“主机名、NSS 与 DNS 名称解析”；防火墙策略属于“firewalld 与完整服务访问链”。本章只读取这些前置事实，不复制相邻章节。

② [知识点] 主机身份验证发生在用户认证之前

建立加密传输后，服务器出示主机公钥并证明持有对应私钥。客户端将收到的主机密钥与可信记录比较：首次连接时通常没有记录，需要通过独立可信渠道核对指纹；后续连接应与既有记录一致；若密钥变化，客户端必须把它视为身份异常，而不是自动接受。

主机身份验证解决的是中间人问题。即使用户密码或私钥完全正确，也不应把凭据交给一个身份未确认的服务器。

③ [知识点] 用户认证可能有多种方法并按次序尝试

服务器公布当前允许继续尝试的方法，例如 `publickey`、`password` 或 `keyboard-interactive`。客户端按配置和可用凭据尝试。公钥失败后若密码仍允许，客户端可能继续提示账号密码并最终登录成功。因此“没有输入账号密码”不是唯一判断，“成功进入 Shell”也不能证明一定使用了公钥；应读取 `ssh -vvv` 中最终的 `Authenticated ... using "publickey"` 等证据，或在测试时显式禁止回退。

④ [知识点] 认证完成后才创建交互 Shell 或远程命令

以下两条命令都建立 SSH 会话，但请求的远端行为不同：

```
ssh operator@servera
ssh operator@servera 'id && hostname'
```

第一条请求交互 Shell；第二条执行指定命令并把标准输出、标准错误和退出状态传回本地。考试验收更适合使用第二种形式，因为它可以明确证明远端用户和主机身份，而不依赖提示符外观。

[Cheatsheet] `timeout/refused` 属于传输层；`host key` 提示属于服务器身份层；`Permission denied` 属于用户认证层；进入 Shell 后仍要用 `id`、`hostname` 或指定命令确认功能对象。

[知识专题] 主机密钥与 `known_hosts`：客户端如何记住服务器是谁

主机密钥不是登录用户的密钥。它属于 SSH 服务端实例，通常在系统安装或 OpenSSH 服务初始化时生成。客户端保存的是主机公钥或其哈希化主机名记录，用于检测之后连接到的服务器身份是否变化。

① [知识点] 服务端主机密钥与用户密钥是两条独立信任链

典型主机密钥文件位于：

```
/etc/ssh/ssh_host_ed25519_key
/etc/ssh/ssh_host_ed25519_key.pub
/etc/ssh/ssh_host_rsa_key
/etc/ssh/ssh_host_rsa_key.pub
```

没有 `.pub` 后缀的是服务端主机私钥，必须只由特权进程读取；带 `.pub` 的文件可用于通过控制台或其他可信渠道发布指纹。用户自己的 `~/.ssh/id_*` 不应替代主机密钥，主机密钥也不应复制到用户的 `authorized_keys`。

② [查询] 在服务端核对主机公钥指纹

在能够可信访问服务器本地控制台的前提下，可读取某个主机公钥的指纹：

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
ssh-keygen -lf /etc/ssh/ssh_host_rsa_key.pub
```

客户端首次连接时显示的算法和 SHA256 指纹应与服务器本地证据一致。电话、受控工单、控制台画面或已经可信的管理渠道都可作为带外核验方式。`ssh-keyscan` 只能从网络抓取当前对端提供的公钥，不能独立证明这个对端就是预期服务器，因此不能把“抓到的值”和“可信身份”混为一谈。

③ [知识点] `known_hosts` 有用户级和系统级两个入口

常见文件为：

```
~/ssh/known_hosts
/etc/ssh/ssh_known_hosts
```

用户级文件只影响当前用户；系统级文件可由管理员预置组织信任。条目通常包含主机标识、密钥算法和公钥材料。启用主机名哈希时，肉眼不再能直接搜索主机名，但 `ssh-keygen -F` 仍可查找。

非默认端口通常以 `[host]:port` 作为记录目标，例如：

```
ssh-keygen -F '[servera.example.com]:2222'
```

④ [操作] 查询和删除指定主机记录，而不是清空整个文件

```
ssh-keygen -F servera.example.com
ssh-keygen -R servera.example.com
ssh-keygen -R '[servera.example.com]:2222'
```

`-F` 用于查询；`-R` 删除与指定目标匹配的条目，并通常保留备份。删除操作不是验证手段。正确顺序是：先确认服务器确实重装、迁移或轮换了主机密钥；再通过可信渠道核对新指纹；最后删除旧条目并重新连接。

⑤ [边界] 不使用“关闭检查”处理主机密钥异常

`StrictHostKeyChecking=no`、把 `UserKnownHostsFile` 指向 `/dev/null`，或直接删除整个 `known_hosts`，都会降低或消除身份连续性。这些配置可能用于明确受控的临时场景，但不应成为考试和日常排错的默认答案。若出现 `REMOTE HOST IDENTIFICATION HAS CHANGED`，最小安全动作是停止连接、核对变更原因和新指纹，再只更新相关条目。

[Cheatsheet] 主机私钥留在服务端；客户端保存主机公钥记录；首次连接先核验指纹；变化时先调查再 `ssh-keygen -R`；`ssh-keyscan` 是采集工具，不是独立信任根。

[操作专题] 用户密钥对：生成、保护、识别和轮换

用户密钥对用于证明客户端持有某项身份。私钥承担签名能力，公钥承担可分发的验证材料。稳定操作应显式指定算法和文件路径，避免覆盖既有默认密钥，也避免依赖不同 RHEL 9 次版本或安全模式下可能变化的默认算法。

① [操作] 生成专用密钥对并明确文件名

非 FIPS 环境可使用 Ed25519：

```
ssh-keygen -t ed25519 -f ~/.ssh/rhcsa20_operator -C 'operator@workstation'
```

需要兼容 FIPS 或明确要求 RSA 时，可使用：

```
ssh-keygen -t rsa -b 3072 -f ~/.ssh/rhcsa20_operator -C 'operator@workstation'
```

命令生成两个文件：

```
~/ssh/rhcsa20_operator      私钥
~/ssh/rhcsa20_operator.pub  公钥
```

`-t` 选择算法，`-b` 对适用算法设置位数，`-f` 指定输出路径，`-C` 写入便于识别的注释。实际使用前先确认目标路径不存在，避免覆盖仍在其他服务器上生效的私钥。

② [知识点] 密语与文件权限解决不同风险

私钥密语加密私钥内容，即使文件副本泄露，攻击者仍需解锁密语；文件权限限制本机其他账号读取。两者应同时考虑。典型权限为：

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/rhcsa20_operator
chmod 644 ~/.ssh/rhcsa20_operator.pub
```

公钥不要求保密，但仍应防止被无意改写。私钥权限过宽时，OpenSSH 客户端可能拒绝使用它。

③ [操作] 用指纹识别密钥而不是只看文件名

```
ssh-keygen -lf ~/.ssh/rhcsa20_operator.pub
ssh-keygen -y -f ~/.ssh/rhcsa20_operator | ssh-keygen -lf -
```


第一条读取公钥指纹；第二组命令从私钥导出公钥后计算指纹，可用于确认一对文件是否对应。执行第二组命令可能要求输入私钥密语。不要把私钥内容输出到终端、聊天、工单或共享目录。

④ [操作] 改变私钥密语而不生成新的身份

```
ssh-keygen -p -f ~/.ssh/rhcsa20_operator
```

这会重新保护同一私钥，公钥身份和指纹不变。若任务要求真正轮换身份，应生成新密钥对、先把新公钥部署并验证，再移除旧授权；不能把“改密语”当成“轮换公钥”。

⑤ [安全边界] 丢失私钥与泄露私钥的处理不同

私钥丢失且没有备份时，无法从公钥恢复私钥，应通过其他可信管理入口部署新公钥。私钥可能泄露时，必须把对应公钥从所有服务器的授权位置撤销，并生成新身份；仅修改本地文件权限或密语不足以撤销已经复制出去的私钥副本。

[Cheatsheet] `-f` 避免覆盖；私钥 600、公钥可读；指纹识别身份；改密语不改指纹；泄露时撤销远端公钥，丢失时通过其他入口部署新公钥。

[操作专题] `authorized_keys` 与 `ssh-copy-id`：把公钥变成账号授权

公钥文件存在于客户端并不等于远端账号已授权。服务端必须在目标账号的授权文件中出现匹配公钥，而且路径、所有权、权限、SELinux 标签和 `sshd` 有效策略都必须允许读取。

① [知识点] `authorized_keys` 的一行是一项授权记录

典型行由可选限制、密钥类型、Base64 公钥主体和注释构成：

```
from="192.0.2.0/24",restrict ssh-ed25519 AAAA... operator@workstation
```

普通考试任务通常只需写入“密钥类型 + 公钥主体 + 注释”。`from=`、`command=`、`restrict` 等属于更细的授权约束，理解其位置即可，不在本章展开复杂堡垒机策略。注释不参与加密验证，可用于标识来源和轮换批次。

② [操作] 使用 `ssh-copy-id` 安全追加指定公钥

```
ssh-copy-id -i ~/.ssh/rhcsa20_operator.pub operator@servera
```

`-i` 应指向公钥文件。`ssh-copy-id` 通常通过现有密码或其他已可用认证方式登录远端，并把尚未存在的公钥追加到目标账号的默认授权位置。它不会把私钥复制到服务器，也不会自动修改 `PasswordAuthentication` 或 `PubkeyAuthentication`。

③ [操作] 手工部署时先创建目录，再追加而不是覆盖

在无法使用 `ssh-copy-id`、但已有可信管理入口时，可在服务端以目标用户身份执行：

```
install -d -m 700 ~/.ssh
cat /tmp/operator.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

这里的 `/tmp/operator.pub` 只是受控传入的公钥示例。不要使用 `>` 覆盖已有 `authorized_keys`，除非任务明确要求重建且已保存现有授权。写入后应删除临时公钥副本，并核对重复行和注释。

④ [验证] 强制只测试公钥认证，防止密码回退造成假成功

```
ssh -i ~/.ssh/rhcsa20_operator \
-o IdentitiesOnly=yes \
-o PreferredAuthentications=publickey \
-o PasswordAuthentication=no \
-o KbdInteractiveAuthentication=no \
operator@servera 'id && hostname'
```

此命令显式选择私钥并禁止两类交互式口令回退。成功后仍应确认远端 `id` 和 `hostname`，并用 `ssh -vvv` 查看最终认证方法。

⑤ [边界] 授权文件中有公钥仍可能被拒绝

高频原因包括：连接的用户名错误；用户 `home`、`.ssh` 或 `authorized_keys` 所有权不正确；路径可被其他用户写入；SELinux 标签异常；`AuthorizedKeysFile` 指向其他位置；`Match` 块对该用户关闭公钥认证；账号被锁定、Shell 不可用或被 `AllowUsers` / `DenyUsers` 拒绝。调查时应读取有效配置和服务端日志，而不是重复复制同一公钥。

[Cheatsheet] `ssh-copy-id -i` 指向 `.pub`；授权是追加不是覆盖；测试时禁用密码回退；“公钥已写入”只证明文件内容，不证明 `sshd` 会读取和接受。

[操作专题] 客户端配置与匹配：让别名、用户、端口和身份可预测

当命令行开始反复出现 `-p`、`-i`、`-o` 和长主机名时，应把稳定连接参数写入客户端配置。但配置文件不是简单的“最后一行覆盖”，OpenSSH 对大多数参数采用最先获得的值，因此规则顺序直接影响结果。

① [知识点] 客户端配置来源按优先级读取

主要来源为：

```
命令行选项
→ ~/.ssh/config
→ /etc/ssh/ssh_config 及其 Include
```

对于大多数参数，一旦取得值，后续来源不再覆盖。因此用户配置可以覆盖系统默认，命令行可以覆盖用户配置；同一配置文件中，应把更具体的 `Host` 块放在通用 `Host *` 之前。

② [操作] 为明确目标建立别名

```
Host servera-operator
    HostName servera.example.com
    User operator
    Port 22
    IdentityFile ~/.ssh/rhcsa20_operator
    IdentitiesOnly yes

Host *
    ServerAliveInterval 60
```

`Host` 是客户端匹配模式或别名；`HostName` 才是实际连接目标；`User` 指定远端账号；`Port` 指定服务端端口；`IdentityFile` 指定候选私钥；`IdentitiesOnly yes` 限制客户端只使用显式配置的身份，而不是把 agent 中所有密钥都依次尝试。

③ [验证] 保护用户配置文件并检查最终值

```
chmod 600 ~/.ssh/config
ssh -G servera-operator | grep -E '^(hostname|user|port|identityfile|identitiesonly) '
```

`ssh -G` 在完成 `Host` 和 `Match` 计算后输出有效客户端配置。它能证明客户端将使用哪些参数，但不能证明服务端在线、主机密钥可信或用户认证会成功。

④ [知识点] 命令行 `-o` 适合一次性验证，不应替代长期配置

```
ssh -o IdentitiesOnly=yes -o PasswordAuthentication=no servera-operator
```

`-o key=value` 可快速覆盖或测试单项配置，尤其适合排错和验收。若相同选项长期重复使用，应回到 `~/.ssh/config`，并用 `ssh -G` 验证，避免多个脚本各自携带不一致参数。

⑤ [诊断] 多把密钥时先确认“客户端选择了什么”

客户端可能从默认文件、配置中的多个 `IdentityFile`、PKCS#11 设备和 agent 获取身份。若服务端在认证尝试上限前始终没看到正确密钥，应检查：

```
ssh -G servera-operator
ssh-add -l
ssh -vvv servera-operator
```

重点寻找实际读取的配置文件、`identityfile` 列表、`Offering public key` 和最终认证方法。不要先在服务器端反复改权限，因为客户端可能根本没有发送目标公钥。

[Cheatsheet] 命令行最高，用户配置早于系统配置；多数参数 first-value-wins；具体 `Host` 在前，`Host *` 在后；`ssh -G` 看选择，`ssh -vvv` 看执行。

[操作专题] `ssh-agent` 与 `ssh-add`：在本地会话中安全使用受密语保护的私钥

`ssh-agent` 不会把私钥上传到服务器。它在本地运行，通过 Unix socket 接收签名请求，使客户端可以使用已经解锁的私钥，而不必每次重新输入密语。agent 的存在、环境变量和已载入身份是三个需要分别确认的状态。

① [操作] 在当前 Shell 启动 agent 并导入环境

```
eval "$(ssh-agent -s)"
printf '%s\n' "$SSH_AUTH_SOCK"
```

`ssh-agent -s` 输出适合 Bourne Shell 的环境变量设置，`eval` 将其导入当前 Shell。`SSH_AUTH_SOCK` 应指向 agent 的本地 socket。桌面会话、登录管理器或其他工具可能已经启动 agent，重复启动会产生多个彼此独立的身份集合。

② [操作] 载入、列出和移除身份

```
ssh-add ~/.ssh/rhcsa20_operator
ssh-add -l
ssh-add -L
ssh-add -d ~/.ssh/rhcsa20_operator
ssh-add -D
```

`-l` 列出指纹，适合确认载入了哪一把密钥；`-L` 输出公钥；`-d` 删除指定身份；`-D` 删除全部身份。删除 agent 中的身份不会删除磁盘上的私钥文件。

③ [操作] 为缓存设置有限生命周期

```
ssh-add -t 1h ~/.ssh/rhcsa20_operator
```

该身份在约一小时后从 agent 中自动移除。生命周期控制适合临时管理会话，但它不是远端授权撤销：即使 agent 中已删除，私钥文件仍可再次载入；真正撤销访问还需要从服务器删除对应公钥。

④ [安全边界] agent forwarding 不是普通密钥认证的必要条件

`ForwardAgent yes` 允许远端会话借助本地 agent 继续向其他主机认证。远端不会直接得到私钥文件，但被入侵的远端环境可能在会话期间滥用转发的 agent。RHCSA 普通直连任务不需要开启 agent forwarding；需要跳转时优先评估 `ProxyJump` 等不暴露 agent 的方案，并把复杂跳板策略留给专门设计。

[Cheatsheet] agent 是本地签名服务；`SSH_AUTH_SOCK` 证明当前 Shell 能否访问它；`ssh-add -l` 看身份；删除 agent 身份不删除文件，也不撤销服务端公钥。

[知识专题] `sshd` 配置源、加载顺序和连接上下文

服务端故障经常不是“某一行写错”，而是管理员看的文件和守护进程实际使用的有效值不同。RHEL 9 通常在主配置中包含 `/etc/ssh/sshd_config.d/*.conf`，但必须检查当前系统的 `Include` 位置和目录内容。OpenSSH 对大多数关键字采用最先取得的值，因此不能套用其他软件“最后一行覆盖”的习惯。

① [知识点] 守护进程、主机密钥和用户授权是三类对象

`sshd` 负责监听、协商和认证；主机私钥证明服务器身份；目标用户的授权文件决定哪些用户公钥可用。`sshd.service` 为 `active` 不表示主机密钥一定有效，也不表示某个用户的 `authorized_keys` 可读。

② [查询] 先读取实际配置入口和 drop-in 列表

```
grep -nE '^[[:space:]]*'
(Include|Match|Port|PasswordAuthentication|KbdInteractiveAuthentication|PubkeyAuthentication|PermitRootLogin|AuthorizedKeysFile|AllowUsers|
\
/etc/ssh/sshd_config
find /etc/ssh/sshd_config.d -maxdepth 1 -type f -name '*.conf' -printf '%f\n' | sort
```

第一条建立主文件基线并定位 `Include` 与 `Match`；第二条显示候选 drop-in 的字典序。不要在未读取现有文件时创建一个“99-local.conf”并假设数字最大就一定覆盖，因为 `first-value-wins` 可能使早期文件先锁定参数。

③ [知识点] `Match` 让有效值依赖连接上下文

`Match User`、`Match Group`、`Match Address` 等可对特定连接改变部分指令。进入 `Match` 块后，后续允许的指令只作用于匹配连接，直到新的 `Match` 或文件结束。把全局指令误写进条件块，或只运行不带上下文的 `sshd -T`，都可能漏掉目标用户的实际配置。

④ [验证] 使用 `sshd -t`、`-T` 和 `-T -C` 分别回答三个问题

```
sshd -t
sshd -T | grep -E '^(port|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|permitrootlogin|authorizedkeysfile) '
sshd -T -C user=operator,host=servera.example.com,addr=192.0.2.50,laddr=192.0.2.10,lport=22 \
| grep -E '^(pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|permitrootlogin|authorizedkeysfile) '
```

- `sshd -t`：配置能否解析，关键文件是否满足启动检查；
- `sshd -T`：输出全局有效配置；

- `sshd -T -C ...`：按给定连接参数应用 `Match` 后输出有效配置。

静态检查成功不表示守护进程已经重载，也不表示真实网络登录成功。

⑤ [知识点] 认证相关指令必须按目标拆开验证

常见指令包括：

指令	回答的问题
<code>PubkeyAuthentication</code>	是否允许公钥用户认证
<code>PasswordAuthentication</code>	是否允许 SSH password 方法
<code>KbdInteractiveAuthentication</code>	是否允许键盘交互式认证
<code>PermitRootLogin</code>	root 通过 SSH 的允许范围
<code>AuthorizedKeysFile</code>	到哪里读取用户授权公钥
<code>AllowUsers</code> / <code>DenyUsers</code>	哪些用户可进入认证链

“关闭密码登录”不能只凭一行注释判断，也不能把 `PasswordAuthentication no` 自动扩大为所有交互式认证都关闭。应查看有效值和目标连接上下文。

[Cheatsheet] 先找 `Include` 和 `Match`；OpenSSH 多数参数 first-value-wins；`-t` 看可解析，`-T` 看有效值，`-T -C` 看目标连接；三者都不替代真实登录。

[操作专题] 安全修改 `sshd`：先保留入口，再静态检查，再开第二条连接

SSH 是远程管理入口，自身变更比普通服务更需要回退意识。最危险的做法不是某个参数选错，而是在只有一条远程会话时同时关闭现有认证方法、重载服务并退出会话。安全流程必须把基线、最小变更、静态检查、当前状态和独立新连接分开。

① [操作] 变更前记录基线和恢复入口

```
systemctl is-active sshd
ss -lntp | grep -E '(:22\b|sshd)'
```

```
sshd -T > /root/sshd-effective.before
cp -a /etc/ssh/sshd_config.d /root/sshd_config.d.before
```

同时确认至少一个恢复入口：保持当前 SSH 会话不退出；具备 BMC/控制台；或有另一个已验证管理员账号。备份路径只是任务示例，真实环境应遵守本地变更管理要求。

② [操作] 使用最小 drop-in 表达目标状态

例如建立一个内容明确的本地文件：

```
# /etc/ssh/sshd_config.d/10-local-auth.conf
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

文件名前缀必须结合现有目录和 first-value-wins 规则选择。写入后不立即 reload，先读取文件并运行静态检查。不要复制整份发行版主配置到 drop-in，也不要同时修改多个来源而无法判断谁生效。

③ [验证] 先检查语法，再检查有效值和目标用户上下文

```
sshd -t
sshd -T | grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication) '
```

```
sshd -T -C user=operator,host=servera.example.com,addr=192.0.2.50,laddr=192.0.2.10,lport=22 \
| grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication) '
```

若有效值与文件内容不同，应调查更早配置来源和 `Match`，而不是继续增加重复指令。

④ [操作] 使用 reload 应用后检查服务和监听

```
systemctl reload sshd
systemctl is-active sshd
ss -lntp | grep sshd
journalctl -u sshd --since '-5 min' --no-pager
```

`reload` 使新连接使用新配置，通常不会终止既有会话。服务仍为 `active` 和端口仍监听只证明守护进程层，不证明目标用户能按预期认证。

⑤ [验证] 从独立客户端开第二条连接，成功后才关闭旧入口

```
ssh -i ~/.ssh/rhcsa20_operator \  
-o IdentitiesOnly=yes \  
-o PreferredAuthentications=publickey \  
-o PasswordAuthentication=no \  
-o KbdInteractiveAuthentication=no \  
operator@servera 'id && hostname'
```

另行测试不应允许的路径，例如 root 直登或 password-only。只有允许路径成功、禁止路径按预期失败、服务端日志无异常后，才退出旧管理会话。若新连接失败，应使用旧会话恢复 drop-in、重新运行 `sshd -t` 并 reload。

[Cheatsheet] 旧会话不退出；最小 drop-in；`-t` 后才 reload；active+listen 仍不是登录终态；必须用第二连接验证允许和禁止路径。

[知识专题] 权限、所有权、`StrictModes` 与 SELinux：为什么“公钥明明在文件里”仍失败

OpenSSH 不只检查公钥内容，还检查授权路径是否可信。若其他用户可以替换目标用户的 home、`.ssh` 或 `authorized_keys`，公钥认证就失去意义。RHEL 的 SELinux 又增加了对象标签这一独立检查层，因此 Unix mode 正确并不保证标签正确，标签正确也不替代所有权和 mode。

① [知识点] 客户端私钥和配置文件必须防止本机他人读取或改写

保守且常用的设置为：

```
chmod 700 ~/.ssh  
chmod 600 ~/.ssh/config  
chmod 600 ~/.ssh/rhcsa20_operator  
chmod 644 ~/.ssh/rhcsa20_operator.pub
```

私钥过宽时客户端通常拒绝使用。配置文件可改变目标、用户、代理和命令，虽然不含私钥，也不应允许其他用户改写。

② [查询] 在服务端逐层检查路径，而不是只看最终文件

```
namei -l /home/operator/.ssh/authorized_keys  
stat -c '%U:%G %a %n' /home/operator /home/operator/.ssh /home/operator/.ssh/authorized_keys
```

重点是：目标用户和组是否正确；`.ssh` 与授权文件是否归目标用户所有；路径是否可被组或其他用户写入。常用保守设置是 `.ssh` 为 700、`authorized_keys` 为 600。home 不必机械改为 700，但不应被不可信用户写入。

③ [知识点] `StrictModes` 检查的是授权路径的可信性

`StrictModes yes` 时，`sshd` 在接受登录前检查用户文件和 home 的所有权、权限。关闭 `StrictModes` 会掩盖根因并降低安全性，不应作为默认修复。更合理的做法是沿路径修正 owner 和可写位，再重新触发一次登录并查看日志。

④ [验证] 检查并恢复默认 home 路径的 SELinux 标签

```
ls -ldZ /home/operator /home/operator/.ssh /home/operator/.ssh/authorized_keys  
restorecon -Rv /home/operator/.ssh
```

`restorecon` 根据现有策略恢复默认标签，适合文件被复制、解压或手工创建后标签异常的情况。不要用 `chcon` 作为长期默认修复，因为其临时标签可能在重新标记时丢失。

⑤ [边界] 非标准 home、NFS home 和自定义端口需要转入专门章节

若 home 位于非标准路径，可能需要持久 `semanage fcontext` 规则；若通过 NFS 提供 home，可能涉及 SELinux Boolean 和网络文件系统语义；若改变 SSH 监听端口，可能涉及 `ssh_port_t` 和 `firewalld`。当前章只建立“权限与标签必须分别验证”的接口，完整规则归“SELinux 文件规则、端口类型与 Boolean”和“firewalld 与完整服务访问链”。

[Cheatsheet] `namei -l` 看每一级路径；`.ssh` 700、授权文件 600 是保守基线；不关闭 `StrictModes`；`ls -Z` 与 mode 分开；默认 home 标签用 `restorecon`，非标准路径转 SELinux 章节。

[诊断专题] 从客户端到服务端：用最有区分度的下一条证据推进

SSH 故障适合严格按阶段调查。每次只提出一个假设，再选能区分该假设的证据。不要同时删除 `known_hosts`、重新生成密钥、改 `sshd_config`、放宽权限和关闭 SELinux，因为这样即使恢复也无法知道真正原因。

① [诊断] 症状：名称无法解析或连接目标错误

当前证据：`ssh -vvv` 在连接前显示的目标名称和地址。假设：客户端别名、`HostName` 或名称解析把连接导向错误服务器。下一条证据：`ssh -G alias` 查看最终 `hostname`、`user`、`port`；再引用第 19 章核对 NSS/DNS。最小修复：修正客户端配置或名称记录，不改密钥。再验证：`ssh -vvv` 中目标地址与预期一致。

② [诊断] 症状：timeout 或 connection refused

当前证据：客户端是否已打印 `Connecting to ... port ...`。假设：`timeout` 偏向路径或过滤；`refused` 偏向目标未监听或端口错误。下一条证据：在服务器本地运行 `systemctl is-active sshd` 与 `ss -lnpt`；防火墙转第 21 章。最小修复：启动或修正监听配置，或修正客户端端口。再验证：客户端进入主机密钥或认证阶段。

③ [诊断] 症状：Host key verification failed 或主机身份已变化

当前证据：警告中显示的算法、指纹和 `offending line`。假设：合法重装/轮换、连接到错误地址，或中间人攻击。下一条证据：通过控制台读取 `/etc/ssh/ssh_host_*.key.pub` 指纹，并核对当前 DNS/IP。最小修复：只有在确认合法变化后，使用 `ssh-keygen -R` 删除特定记录。再验证：新连接显示的新指纹与可信证据一致。

④ [诊断] 症状：客户端没有发送预期公钥

当前证据：`ssh -vvv` 没有目标密钥的 `Offering public key`。假设：`IdentityFile` 未匹配、文件权限错误、agent 无身份、别名匹配了其他 `Host` 块。下一条证据：`ssh -G`、`stat`、`ssh-add -l`。最小修复：显式 `-i` 与 `IdentitiesOnly=yes`，修正客户端配置或载入 agent。再验证：调试输出出现目标指纹的 `offer`。

⑤ [诊断] 症状：客户端发送了公钥，但服务端拒绝

当前证据：`Offering public key` 后仍出现 `Authentications that can continue`，没有接受信息。假设：连接用户名错误；授权文件无匹配公钥；有效配置关闭公钥；路径权限/标签不可信；账号策略拒绝。下一条证据：同时读取：

```
sshd -T -C user=operator,host=servera.example.com,addr=192.0.2.50,laddr=192.0.2.10,lport=22
namei -l /home/operator/.ssh/authorized_keys
ls -ldZ /home/operator/.ssh /home/operator/.ssh/authorized_keys
journalctl -u sshd --since '-5 min' --no-pager
```

最小修复：只修正被证据指向的配置、授权内容、owner/mode 或标签。再验证：`key-only` 连接成功，日志显示目标用户公钥认证完成。

⑥ [诊断] 症状：登录成功但方法、用户或主机不符合任务

当前证据：客户端最终认证方法和远端命令结果。假设：公钥失败后回退密码；客户端 `User` 错误；别名指向错误主机；远端命令在跳转节点执行。下一条证据：`ssh -vvv` 最终 `Authenticated ... using ...`，远端 `id`、`hostname`，客户端 `ssh -G`。最小修复：禁止回退、修正 `User/HostName/IdentityFile`。再验证：指定用户、指定主机、指定认证方法和命令退出状态全部符合要求。

[Cheatsheet] 先定位阶段：`ssh -G` 查选择，`ssh -vvv` 查执行；客户端没 `offer` 就先查客户端；已 `offer` 未接受再查服务端；每次修复后重新触发同一个最小测试。

[知识专题] 工作迁移与安全边界：把考试中的 SSH 变成可维护入口

考试通常只要求在少量主机之间建立连接，真实工作还要考虑轮换、离职撤权、资产重组、批量预置信任和自动化。扩展内容仍围绕本章对象，不展开大型身份平台或证书体系。

① [知识点] 远程入口变更必须设计回退

任何关闭密码、禁止 `root` 或改变端口的操作，都应先证明替代路径可用。保持旧会话、准备控制台、记录有效配置和只修改一个来源，能够把“配置错误”从远程失联事故降级为可回退变更。

② [知识点] 密钥轮换是“先加新、验证、再删旧”

```
生成新密钥对
→ 在服务器追加新公钥
→ 使用新私钥完成 key-only 验证
→ 从所有服务器删除旧公钥
→ 记录旧指纹已撤销
```

直接覆盖本地私钥或先删除旧公钥，会在任一步失败时失去入口。注释和指纹应作为轮换清单的稳定标识。

③ [知识点] 批量主机信任需要可信分发，不是盲目扫描

组织可通过配置管理分发 `/etc/ssh/ssh_known_hosts`，但输入的主机公钥必须来自可信资产流程。`ssh-keyscan` 可帮助收集格式化密钥，却不能在不可信网络上自行解决身份认证。首次信任的来源决定了之后检测变化是否有意义。

④ [知识点] 为 Ansible 准备的是稳定状态，不是把私钥散发到受管主机

后续 RHCE 自动化通常由控制节点发起 SSH。应明确控制节点上的用户、私钥、known_hosts 和 agent 状态，以及受管节点上的账号、authorized_keys、sudo 和 sshd 有效策略。私钥留在控制节点；受管节点只接收公钥。Inventory、ansible_user、私钥字段和批量幂等配置属于 RHCE 章节，本章提供可验证的手工状态模型。

[Cheatsheet] 变更先有回退；轮换先加后删；批量信任需要可信来源；控制节点持私钥，受管节点持公钥；不要把“能连”扩大为“身份和权限都正确”。

[经典任务] 任务一：为指定用户建立可审计的密钥认证

环境与当前状态

- 客户端：workstation.example.com，当前本地用户 student；
- 服务端：servera.example.com；
- 目标远端账号：operator1；
- operator1 当前可以使用账号密码登录；
- 客户端尚无本任务专用密钥和别名；
- 服务端主机密钥指纹可通过服务器控制台取得；
- 本任务不要求改变 sshd 全局认证策略。

目标终态

1. 在客户端生成专用密钥：

```
~/ .ssh/rhcsa20_operator
~/ .ssh/rhcsa20_operator.pub
```

2. 私钥必须使用密语保护；不覆盖其他密钥；
3. 首次信任 servera 前，通过控制台核对主机指纹；
4. 将公钥追加到 operator1 的授权文件；
5. 在 ~/.ssh/config 建立别名 servera-operator，显式设置 HostName、User、IdentityFile 和 IdentitiesOnly yes；
6. 在当前登录会话中使用 ssh-agent 载入专用私钥；
7. 执行 ssh servera-operator 'id && hostname' 时，必须使用公钥认证且不得回退到密码；
8. 不得把私钥复制到服务端，不得关闭主机密钥检查。

验收证据

- 专用公钥指纹；
- ssh-add -l 中的专用密钥指纹；
- ssh -G servera-operator 的有效目标、用户和身份文件；
- ssh -vvv 中主机身份、密钥 offer、服务端接受和最终认证方法；
- 远端 id 与 hostname；
- 服务端 .ssh、authorized_keys 的 owner、mode 和 SELinux 标签。

[经典任务] 任务二：安全修复 sshd 配置并避免锁死

环境与当前状态

- 管理员已通过一条现有 SSH 会话连接到 serverb.example.com；
- operator1 的公钥已经写入服务器，但登录仍失败；
- /etc/ssh/sshd_config.d/ 中存在本地 drop-in，至少一项认证配置无效或未按预期生效；
- .ssh 路径的所有权、权限或 SELinux 标签可能异常；
- sshd 当前仍在运行并监听默认端口；
- 管理员可以另开一个客户端终端，但不能依赖服务器重启。

目标策略

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

`operator1` 必须能够使用指定私钥登录；`root` 直登和口令式登录必须按策略失败。

限制条件

- 不关闭当前管理会话；
- 不停止 `firewalld`，不关闭 `SELinux`；
- 不删除全部现有 SSH 配置；
- 不在 `sshd -t` 通过前 `reload`；
- 不用 `chmod 777`、关闭 `StrictModes` 或 `StrictHostKeyChecking=no` 作为修复；
- 新连接未验证成功前不得退出旧会话。

验收证据

- 配置源与 `Include`、`Match` 基线；
- `sshd -t`；
- `sshd -T` 和目标用户的 `sshd -T -C ...`；
- `systemctl is-active sshd` 与 `ss -lntp`；
- `namei -l`、`stat`、`ls -Z`；
- 独立客户端的 `key-only` 成功验证；
- `root` 和 `password-only` 的预期失败；
- `journalctl -u sshd` 的服务端证据。

[参考解答] 任务一：为指定用户建立可审计的密钥认证

1. 建立客户端基线

先检查现有目录和目标文件，避免覆盖：

```
install -d -m 700 ~/.ssh
ls -l ~/.ssh/rhcsa20_operator ~/.ssh/rhcsa20_operator.pub 2>/dev/null
```

若目标文件已存在，应先确认是否属于旧任务或正在使用。除非题目明确要求重建，不直接覆盖。

2. 生成受密语保护的专用密钥

非 FIPS 环境可选择 Ed25519：

```
ssh-keygen -t ed25519 \
-f ~/.ssh/rhcsa20_operator \
-C 'student@workstation-rhcsa20'
```

若环境处于 FIPS 模式或题目要求 RSA：

```
ssh-keygen -t rsa -b 3072 \
-f ~/.ssh/rhcsa20_operator \
-C 'student@workstation-rhcsa20'
```

按提示设置非空密语。随后验证：

```
stat -c '%U:%G %a %n' ~/.ssh/rhcsa20_operator ~/.ssh/rhcsa20_operator.pub
ssh-keygen -lf ~/.ssh/rhcsa20_operator.pub
```

预期私钥只有当前用户可读写，公钥指纹可作为后续 `agent` 和远端授权核对依据。

3. 首次连接前核对服务端主机指纹

在服务端可信控制台读取实际启用的主机公钥指纹，例如：

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

客户端发起一次连接并停在首次信任提示：

```
ssh operator1@servera.example.com
```


只有显示的算法和 SHA256 指纹与控制台证据一致时才接受。若不一致，停止任务并调查名称、地址或主机重装情况。

4. 部署公钥

使用已有密码路径：

```
ssh-copy-id -i ~/.ssh/rhcsa20_operator.pub operator1@servera.example.com
```

注意 `-i` 指向 `.pub` 文件。完成后在服务端检查：

```
ssh operator1@servera.example.com \  
'namei -l ~/.ssh/authorized_keys; stat -c "%U:%G %a %n" ~/.ssh ~/.ssh/authorized_keys; ls -ldZ ~/.ssh ~/.ssh/authorized_keys'
```

此时使用密码进入服务器只用于检查文件状态，不能作为公钥成功证据。

5. 建立客户端别名

编辑 `~/.ssh/config`，添加：

```
Host servera-operator  
  HostName servera.example.com  
  User operator1  
  IdentityFile ~/.ssh/rhcsa20_operator  
  IdentitiesOnly yes
```

然后：

```
chmod 600 ~/.ssh/config  
ssh -G servera-operator | grep -E '^(hostname|user|port|identityfile|identitiesonly) '
```

应确认 `hostname` 为服务端 FQDN，`user` 为 `operator1`，`identityfile` 为专用私钥。

6. 启动或复用 agent 并载入身份

先看当前 Shell 是否已有 agent：

```
printf '%s\n' "$SSH_AUTH_SOCK"  
ssh-add -l
```

若不可达，再启动：

```
eval "$(ssh-agent -s)"
```

载入专用密钥并核对指纹：

```
ssh-add -t 1h ~/.ssh/rhcsa20_operator  
ssh-add -l
```

`ssh-add -l` 中应出现与 `ssh-keygen -lf ~/.ssh/rhcsa20_operator.pub` 一致的指纹。

7. 执行不允许回退的功能验证

```
ssh -vvv \  
-o PreferredAuthentications=publickey \  
-o PasswordAuthentication=no \  
-o KbdInteractiveAuthentication=no \  
servera-operator 'id && hostname'
```

检查要点：

1. 连接目标和端口正确；
2. 主机密钥与已知记录匹配；
3. 出现专用密钥的 `Offering public key`；
4. 服务端接受该密钥；
5. 最终认证方法为 `publickey`；
6. `id` 显示 `operator1`，`hostname` 显示 `servera` 的预期名称；

7. 命令退出状态为 0。

8. 典型错误与最小修复

症状	先查证据	最小修复
ssh-copy-id 提示文件无效	file、文件名是否以 .pub 结尾	改为公钥文件，不复制私钥
能登录但提示了账号密码	ssh -vvv 最终方法	禁止回退后重新验证，调查公钥拒绝原因
没有 offer 专用密钥	ssh -G、ssh-add -l	修正别名、IdentityFile 或载入 agent
host key changed	控制台指纹、DNS/IP	确认合法变更后只删除目标记录
服务端拒绝公钥	journal、owner/mode/label	只修正被证据指出的路径或策略

任务完成后，不需要从服务端删除公钥；它正是目标持久状态。可按要求从 agent 移除临时身份：

```
ssh-add -d ~/.ssh/rhcsa20_operator
```

[参考解答] 任务二：安全修复 sshd 配置并避免锁死

1. 保留现有会话并建立恢复能力

当前 SSH 会话保持打开。确认另一个客户端终端可用，并记录当前服务与监听：

```
systemctl is-active sshd
systemctl status sshd --no-pager
ss -lntp | grep sshd
```

若已有 BMC 或控制台，确认其可用。不要在只有一条不可恢复会话时继续关闭认证方法。

2. 读取配置源而不是先写新文件

```
grep -nE '^[[:space:]]*(Include|Match|Port|PermitRootLogin|PubkeyAuthentication|PasswordAuthentication|KbdInteractiveAuthentication|AuthorizedKeysFile|AllowUsers|
\
/etc/ssh/sshd_config

find /etc/ssh/sshd_config.d -maxdepth 1 -type f -name '*.conf' -print | sort

for f in /etc/ssh/sshd_config.d/*.conf; do
[ -e "$f" ] || continue
printf '\n### %s\n' "$f"
grep -nE '^[[:space:]]*(Match|Port|PermitRootLogin|PubkeyAuthentication|PasswordAuthentication|KbdInteractiveAuthentication|AuthorizedKeysFile|AllowUsers|DenyUser
"$f"
done
```

建立备份和有效配置基线：

```
cp -a /etc/ssh/sshd_config /root/sshd_config.before-rhcsa20
cp -a /etc/ssh/sshd_config.d /root/sshd_config.d.before-rhcsa20
sshd -T > /root/sshd-effective.before-rhcsa20
```

3. 先运行静态检查定位无效内容

```
sshd -t
```

若返回错误，按提示定位文件和行号。只修改错误参数或语法，不删除其他管理员配置。修复后重复 sshd -t，直到无错误。

4. 检查目标用户的授权路径

```
getent passwd operator1
namei -l /home/operator1/.ssh/authorized_keys
stat -c '%U:%G %a %n' \
/home/operator1 \
/home/operator1/.ssh \
/home/operator1/.ssh/authorized_keys
ls -ldZ \
/home/operator1 \
/home/operator1/.ssh \
/home/operator1/.ssh/authorized_keys
```

在默认本地 home 场景，可进行保守修复：

```
chown -R operator1:operator1 /home/operator1/.ssh
chmod 700 /home/operator1/.ssh
chmod 600 /home/operator1/.ssh/authorized_keys
restorecon -Rv /home/operator1/.ssh
```

不机械改变整个 home 为 700；只确保路径不被不可信用户写入。若 home 为非标准路径或 NFS，停止套用默认修复并转入 SELinux/NFS 专题调查。

5. 建立最小认证策略 drop-in

先根据目录顺序和 first-value-wins 判断合适文件名。假设检查后确认 10-local-auth.conf 会最先为这些参数提供本地值，可写入：

```
cat > /etc/ssh/sshd_config.d/10-local-auth.conf <<'EOF'
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
```

再次静态检查：

```
sshd -t
```

6. 验证全局和目标连接的有效配置

```
sshd -T | grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|authorizedkeysfile) '

sshd -T -C user=operator1,host=serverb.example.com,addr=192.0.2.50,laddr=192.0.2.20,lport=22 \
| grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|authorizedkeysfile) '
```

这里的地址应替换为题目真实客户端和服务端地址。若输出仍不是目标值，说明更早配置或 Match 块正在生效，应回到第 2 步调查，而不是添加更多重复行。

7. reload 并验证守护进程层

```
systemctl reload sshd
systemctl is-active sshd
ss -lntp | grep sshd
journalctl -u sshd --since '-5 min' --no-pager
```

若 reload 失败，保持旧会话，读取日志，恢复备份或修正 drop-in；不要重启服务器来掩盖配置错误。

8. 从第二个终端验证允许路径

```
ssh -vvv \
-i ~/.ssh/rhcsa20_operator \
-o IdentitiesOnly=yes \
-o PreferredAuthentications=publickey \
-o PasswordAuthentication=no \
-o KbdInteractiveAuthentication=no \
operator1@serverb.example.com 'id && hostname'
```

必须确认：

- 主机身份记录匹配；
- 客户端发送目标公钥；

- 服务端接受；
- 最终方法为 `publickey`；
- 远端用户为 `operator1`；
- 主机为 `serverb`。

9. 验证禁止路径

验证 `root` 直登不被允许：

```
ssh -o PreferredAuthentications=publickey,password,keyboard-interactive \
root@serverb.example.com
```

验证 `operator1` 不能只用口令方法：

```
ssh \
-o PubkeyAuthentication=no \
-o PreferredAuthentications=password,keyboard-interactive \
operator1@serverb.example.com
```

预期结果是认证失败。该负向测试只能在客户端方法限制、`sshd -T -C` 有效配置和服务端日志三者一致时证明策略终态；不能只看到 `Permission denied` 就结束。

10. 关联服务端日志并收束

在服务器旧会话中：

```
journalctl -u sshd --since '-10 min' --no-pager
```

将允许路径和禁止路径的时间、用户、来源地址和结果对应起来。全部验收完成后，才关闭原管理会话。保留的备份按组织变更流程处理，不在未确认回退窗口结束前删除。

典型错误

错误	为什么危险	正确处理
修改后直接 <code>restart sshd</code> 并退出会话	失去唯一入口	先 <code>-t</code> ，用 <code>reload</code> ，保持旧会话并开第二连接
只看 <code>drop-in</code> 文本	<code>first-value-wins</code> 和 <code>Match</code> 可能改变有效值	用 <code>sshd -T/-T -C</code>
只设置 <code>PasswordAuthentication no</code>	<code>keyboard-interactive</code> 可能仍允许口令式认证	分别检查两个有效值
<code>chmod 777 ~/.ssh</code>	授权路径不再可信	正确 owner，目录 700、文件 600
关闭 SELinux 或 <code>StrictModes</code>	掩盖根因并降低安全	查日志、mode、owner、label 后最小修复
服务 <code>active</code> 就宣布完成	<code>active</code> 不证明新登录	独立客户端验证允许和禁止路径

[本章收束] 把 SSH 还原成一条可证明、可回退的身份链

处理 SSH 时，先不要问“该改哪个配置”，而要把当前事实放回正确层次：客户端最终选中了哪个目标；服务器用哪把主机密钥证明身份；客户端实际发送了哪个用户身份；服务端对这个用户和来源应用了什么有效策略；授权路径是否同时满足内容、所有权、权限和标签；最后，独立新连接能否以指定方法到达指定用户和主机。按照这条链推进，才能让每次修复只改变一个被证据指向的对象。

可执行工作方法

1. 用 `ssh -G` 建立客户端选择基线。
2. 用 `ssh -vvv` 定位失败阶段，只摘取关键状态。
3. 主机密钥异常先带外核验，不先删除信任记录。
4. 客户端没有 offer 目标密钥时，只查客户端身份选择。
5. 已 offer 但未接受时，转查 `sshd -T -C`、授权路径和日志。
6. 修改 `sshd` 前保留旧会话，`sshd -t` 通过后才 `reload`。
7. 从第二终端分别验证允许路径和禁止路径。
8. 用远端 `id`、`hostname` 和退出状态确认终态。

章末检查清单

- 主机身份与用户身份没有混写；

- 私钥只保留在客户端或控制节点；
- `known_hosts` 更新前已经核验合法变化；
- `ssh-copy-id -i` 指向公钥文件；
- 客户端有效配置已经用 `ssh -G` 核对；
- 公钥测试禁止了 `password` 和 `keyboard-interactive` 回退；
- `sshd -t` 先于 `reload`；
- `sshd -T -C` 覆盖了目标连接的 `Match` 上下文；
- `owner`、`mode` 与 SELinux `label` 分别检查；
- 旧管理会话在第二连接验收前保持可用；
- `active`、`listen` 与真实登录没有被扩大为同一个结论。

主要判断表

证据或症状	能证明什么	不能证明什么	下一条高区分度证据
<code>ssh -G alias</code>	客户端最终目标、用户、端口和身份选择	服务器在线或会接受认证	<code>ssh -vvv alias</code>
<code>Connection refused</code>	目标返回了端口拒绝	一定是 <code>firewalld</code> 或 <code>sshd</code> 唯一问题	服务端 <code>ss -lntp</code>
主机密钥变化警告	当前对端密钥与记录不一致	新密钥一定恶意或一定合法	带外主机指纹与目标地址
<code>Offering public key</code>	客户端发送了某把候选公钥	服务端已授权或已接受	服务端 <code>journal</code> 与授权路径
<code>sshd -t</code> 无输出	配置可解析并通过基本检查	已 <code>reload</code> 、目标 <code>Match</code> 正确或能登录	<code>sshd -T/-T -C</code>
<code>systemctl is-active sshd</code>	守护进程当前处于 <code>active</code>	端口对外可达或用户认证成功	<code>ss -lntp</code> 与独立客户端测试
<code>authorized_keys</code> 有目标公钥	授权文本存在	路径可信、标签正确或 <code>sshd</code> 会读取	<code>namei -l</code> 、 <code>ls -Z</code> 、 <code>sshd -T -C</code>
<code>key-only</code> 新连接成功	指定用户公钥链可用	禁止路径也符合目标策略	<code>root/password-only</code> 负向验证

向下一章交接

本章已经证明本机 `sshd` 的有效配置、运行状态和监听入口，也给出了从独立客户端发起连接的功能测试。下一章“`firewalld` 与完整服务访问链”将在此基础上继续回答：监听端口怎样关联 `zone`、`service`、`runtime/permanent` 规则，以及为什么“本机正在监听”仍不等于外部流量一定能到达。遇到外部 `timeout` 时，不关闭防火墙作为捷径，而是把 SSH 已建立的监听证据交给下一章继续调查。